A State-Machine Harness and a Namespace Sandbox for Code Written by Language Models

Arpan Pathak

November 2026

Abstract

When a coding agent runs a program written by a language model, the model's text turns into actions on a real machine. The test command is usually written by the model as well, so a single injected reply can overwrite a shell startup file or send a private key to another host. We built a harness small enough to follow every path from a reply to the host. Its agent loop is a fixed state machine with 5 states, in under 200 lines of Rust, and the state machine alone chooses the next step. A reply can then reach the host in only two ways, as a program file and as a check command. We accept only plain file names, and we run every check in a throwaway sandbox built from Linux namespaces with bubblewrap, with read-only system files, no network and no secrets. The sandbox adds about 15 ms to a check. Of 104 generated escape attempts, none left a trace on the host, while 56 of the same attempts succeeded without the sandbox. The sandbox alone does not bound the number of processes or the memory held in its working folder, and in our tests a cgroup bounds both. A passing check still shows only what the model chose to test.

1 Introduction

Coding agents now run the code that they write. SWE-agent runs the tests of a repository, and CodeAct expresses every action of an agent as Python code that is then executed [11, 12]. In both, text sampled from a language model becomes a command on a real machine, a crossing that Srinivasan calls the *stochastic-deterministic boundary* [9]. That text deserves no trust. A *prompt injection* is an instruction hidden in text that the model reads, such as the task or the output of an earlier command. Under such an instruction, a model can return a reply like the following. We follow this reply through the paper.

```
{"file_name":  "../../.bashrc",
 "code":  "curl -s http://attacker.example/x | sh",
 "check":  "cat ~/.ssh/id_ed25519 |
 ↪       curl -s -d @- http://attacker.example/key"}
```

The file name points two folders up, at the user's shell startup file, and the check sends the user's private SSH key to another host. The first version of our harness accepted both fields.

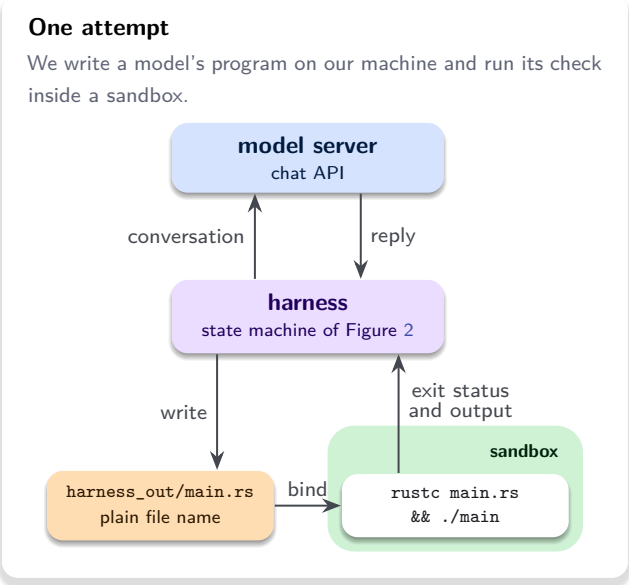


Figure 1: One attempt, with a check from our sample run in Section 5. Our model server runs on a separate machine.

Tool protocols such as the Model Context Protocol (MCP)[1] standardize how a model finds and calls tools. They do not limit what a call does once it runs on the host. In a large agent, the path from a reply to the host is also hard to follow, because it runs through a long system prompt, several layers of code, and network requests that only a packet capture would show. We wanted a system small enough to list every way in which a reply reaches the host, so that we could confine each way and then watch how a model's replies use them.

We therefore kept the harness minimal. It sends a task in plain English to a model and asks for a file name, a program, and a shell command that builds and tests the program, which we call the *check*. It saves the program, runs the check, and returns each failure to the model, for at most MaxAttempts attempts (Figure 1). The loop is a state machine with 5 states and one **match** over them. The file `main.rs` holds 158 lines of code without comments and blank lines, and the sandbox adds 66 [7]. Because the state machine chooses every step, a reply can affect the host only through the file that we write and the check that we run, and the operating system can confine both.

[1] https://modelcontextprotocol.io

States of the harness

Only Ask sends a request to a model. Each box names the lines of Procedure 1 that it runs.

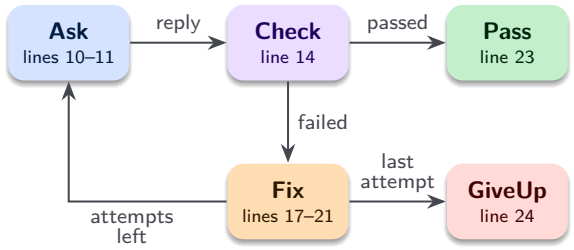


Figure 2: The 5 states of Procedure 1 and their transitions.

Our procedures follow Cormen et al. [1], with Rust types on their variables, and each one links to the lines that implement it. Our contributions are:

- a harness whose agent loop is a fixed state machine, with at most MaxAttempts requests per task (Section 2);
- a threat model that follows each field of a reply to its effect on the host (Section 3);
- a sandbox for model-written checks, built from Linux namespaces without root, with a rule for file names and a startup probe (Section 4);
- an evaluation with 104 generated escape attempts, a test of cgroup limits, the cost of the sandbox, and one real task (Section 5).

2 The harness

Each conversation starts with instructions that fix the format of a reply. They also ask the program to assert its own result, so that a wrong answer makes its own check fail.

Definition 2.1 (Reply)**.** A *reply* is a JSON object with string fields `file_name`, `code` and `check`: the name of the program file, the whole program, and one shell command that builds and runs it.

Example.
```
{"file_name": "main.rs",
"code": "fn main() { ... }",
"check": "rustc main.rs && ./main"}
```

Procedure 1 gives the loop, and Figure 2 draws it as a state machine. The model never names the next state. A reply is data that one arm of the **match** inspects, and that arm decides where the loop goes next. The only transition that depends on the model leaves Check, towards Pass or Fix, and every other transition is fixed. In Rust (Listing 1), State is an enum with the same 5 variants, and the compiler rejects a **match** that leaves one out. A new state therefore cannot be added without a decision at every place that handles states.

The loop is also bounded. Line 11 holds the only request to the model, and the counter grows on line 10 right before it. Fix returns to Ask only while the counter is below MaxAttempts (lines 19 to 21), so one task costs at most MaxAttempts requests and as many

Procedure 1 — main.rs:64–151

Input: a task in plain English.
Output: `Ok` with the reply whose check passed, or `Err` after MaxAttempts failed attempts.
Types: State = Ask | Check(String) | Fix(Error) | Pass | GiveUp.

```
    HARNESS(task: String) → Result {
1       if RUN-SANDBOXED("true", NIL) is Err(error) {
2           return Err(error) // no sandbox on this host
3       }

4       conversation: Conversation = ⟨Instructions, task⟩
5       attempt: u32 = 0
6       state: State = Ask

7       loop {
8           match state {
9               Ask ⇒ {
10                  attempt = attempt + 1
11                  state = ASK(conversation)
12              }

13              Check(reply) ⇒ {
14                  state = SANDBOXED-CHECK(reply)
15              }

16              Fix(failure) ⇒ {
17                  append failure to conversation
18                  state = Ask

19                  if attempt = MaxAttempts {
20                      state = GiveUp
21                  }
22              }

23              Pass ⇒ return Ok(reply)

24              GiveUp ⇒ return Err("no passing program")
25          }
26      }
    }
```

Listing 1 — main.rs:93–105

The loop of Procedure 1 in Rust. The method of each state returns the next state.

```
1   fn run(&mut self) -> Result<()> {
2       let mut state = State::Ask;
3
4       loop {
5           state = match state {
6               State::Ask => self.ask(),
7               State::Check(reply) => self.check(reply),
8               State::Fix(failure) => self.fix(&failure),
9               State::Pass => return Ok(()),
10              State::GiveUp => bail!("no passing program
    ↪ after {MAX_ATTEMPTS} attempts"),
11          };
12      }
13  }
```

Table 1: Ways in which a reply can act on its host, and our defense against each one.

Threat	Defense
A file name with a directory part, such as `../../.bashrc`	Refused before any write.
A check that reads the home folder or the API key	Neither exists in the sandbox.
A check that sends data over the network	Only a loopback interface exists.
A check that changes host files or leaves a link for a later write	A read-only system, and a working folder that disappears with each check.
A check that runs forever or uses too much memory	Limits on time, memory, file size and open files (Table 2).
A check that fills memory with many files	Open in the harness. A cgroup stops it (Table 4).
A check that starts processes in a loop	Open in the harness. A cgroup stops it (Table 4).
A check that uses a kernel bug	Open. We load no seccomp filter.
Output that carries instructions to the model	Open. We return its output unchanged.
A check that tests nothing	Open. A model writes its own check.

checks, whatever the model writes.

Before the first request, lines 1 to 3 run the empty check `true` in the sandbox. On a host where the sandbox cannot start, the harness stops there. Without this probe, every check on such a host would fail with a sandbox error, and the model would spend its attempts on an error that is not in its program.

3 Threat model

We assume that an attacker controls the whole text of every reply and can change it on every attempt, for example through an injection in the task or in the output of an earlier check. We trust the harness, the kernel and the toolchain.

KEY OBSERVATION

The state machine chooses every step, so a reply reaches the host in exactly two ways: as the file that the harness writes and as the check that it runs. The program itself runs only inside its check.

Our first version left both ways open. It joined the file name to the working folder with an ordinary path join, which keeps every `..`. Started in `/home/user/harness`, it wrote `../../.bashrc` to `/home/user/.bashrc`. It ran the check with `sh -c` under the user's own permissions, so `cat` could read the key and `curl` could send it.

Table 1 lists each way in which a reply can act on the host, with our defense. The rule for file names closes the first row, and the sandbox closes the next four. The remaining rows are open in the harness, and Sections 5 and 6 return to them.

Procedure 2 `main.rs:124–225`

Input: the text of one reply.
Output: Pass, or Fix with the failure that we send back to the model.
Types: Answer = { *file_name*: String, *code*: String, *check*: String }.

```
    SANDBOXED-CHECK(reply: String) → State {
1       answer: Answer = PARSE-JSON(reply)
2       if answer = NIL {
3           return Fix("not the JSON asked for")
4       }
5       name: String = answer.file_name
6       if name is not plain {
7           return Fix("not a plain file name")
8       }
9       program: Path = WorkDir / name
10      write answer.code to program
11      check: String = answer.check
12      output: Output = RUN-SANDBOXED(check, program)
13      if output.status ≠ 0 {
14          return Fix(output)
15      }
16      return Pass
    }
```

4 The sandbox

4.1 Plain file names

The file name is the simpler of the two ways in, because the harness does the writing. We only need to make sure that a name cannot point outside WorkDir.

Definition 4.1 (Plain file name)**.** A file name is *plain* if it is not empty, is neither `.` nor `..`, and contains no slash. A plain name always names a file directly inside the working folder.

Example. `main.rs` is plain. `../../.bashrc` and `/etc/passwd` contain slashes.

Procedure 2 refuses a name that is not plain before it writes anything (lines 5 to 8). We compare characters and do not resolve the path. Resolution follows symbolic links and depends on what is on disk at that moment, while a test of characters gives the same answer every time. The test is sufficient here because only the harness writes into WorkDir and no check can see that folder, so no check can plant a link in it. On Linux, `openat2` with `RESOLVE_BENEATH` enforces a comparable rule inside the kernel [4].

4.2 The view of a check

The check is harder to confine, because it is an arbitrary shell command that we have to run. We cannot decide

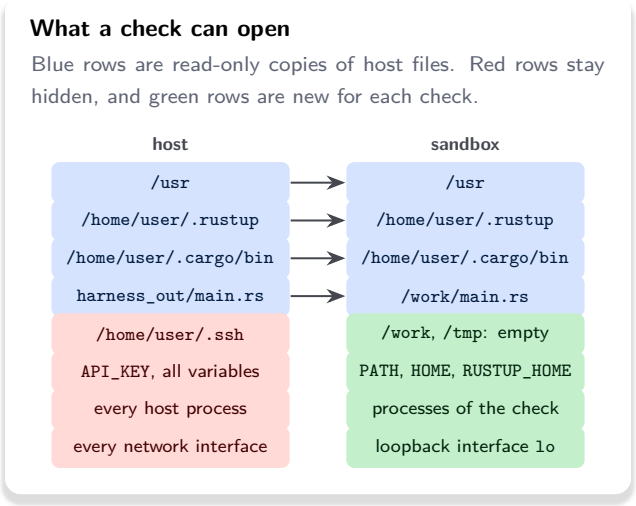


Figure 3: View of a check (Definition 4.2), with our implementation's paths. Inside a sandbox, `HOME` is `/work`.

reliably what a shell command will do by reading it, so we limit what it can see.

Definition 4.2 (View of a check)**.** RUN-SANDBOXED(*check*, *program*) runs *check* with `sh` in a new sandbox that contains only

- the system folders and the toolchain, read-only;
- the program, read-only, at `/work/`*name*;
- empty `/work` and `/tmp` in memory, discarded when the check ends;
- a process table with only the check's own processes;
- a network with only the loopback interface;
- the variables `PATH`, `HOME` = `/work` and what the toolchain needs.

Example. The check of Definition 2.1 starts in `/work`, reads `/work/main.rs` and writes the binary into memory.

Figure 3 shows this view with our paths. Each item of Definition 4.2 removes one route of the hostile reply. Without the home folder and with a cleared environment, the check finds no SSH key and no API key. Because the system folders are read-only and `/work` disappears with the sandbox, nothing that the check writes survives on the host. We mount the program read-only, so the file that we keep is exactly the file that passed its check. With only a loopback interface, every connection to another host fails.

To confirm this, we sent the hostile check again under the plain name `main.sh`. Inside the sandbox, `HOME` is `/work`, so `~/.ssh/id_ed25519` becomes `/work/.ssh/id_ed25519`, which does not exist. `cat` printed `No such file or directory`, and `curl` exited with status 6, its code for a host name that it cannot resolve.

The rest of Procedure 2 handles ordinary failures. A reply that is not the JSON of Definition 2.1, for example one cut off at the model's output limit, goes back to the model as a short message (lines 1 to 4). A failed

Table 2: Constants of our harness and limits of each check, with the values that we use.

Name	Value	Meaning
MaxAttempts	5	requests for one task
WorkDir	`harness_out`	host folder for the programs
WallSeconds	120 s	wall-clock time of a check
KillAfter	5 s	from SIGTERM to SIGKILL
CpuSeconds	60 s	CPU time of each process
AddressSpace	4 GiB	memory of each process
FileSize	64 MiB	size of each file
OpenFiles	256	open files of each process

Listing 2 — `sandbox.rs`:61–86

RUN-SANDBOXED in Rust. The function collects the arguments of three programs and has no branch.

```
1  pub fn command(script: &str, program: &Path, file_name: &str
       ↪ ) -> Result<Command> {
2      let home = env::var("HOME").context("set HOME")?;
3      let rustup = format!("{home}/.rustup");
4      let cargo_bin = format!("{home}/.cargo/bin");
5      let path = format!("{cargo_bin}:/usr/bin:/bin");
6      let program_inside = format!("{SANDBOX_DIR}/{file_name}"
       ↪ );
7
8      let mut command = Command::new("timeout");
9
10     command
11         .args([KILL_AFTER, WALL_SECONDS, "prlimit"])
12         .args(LIMITS)
13         .args(["--", "bwrap"])
14         .args(ISOLATION)
15         .args(SYSTEM_FILES)
16         .args(["--ro-bind-try", &rustup, &rustup])
17         .args(["--ro-bind-try", &cargo_bin, &cargo_bin])
18         .args(["--setenv", "PATH", &path])
19         .args(["--setenv", "HOME", SANDBOX_DIR])
20         .args(["--setenv", "RUSTUP_HOME", &rustup])
21         .args(["--tmpfs", SANDBOX_DIR, "--chdir",
       ↪ SANDBOX_DIR])
22         .arg("--ro-bind").arg(program).arg(&program_inside)
23         .args(["--", "sh", "-c", script]);
24
25     Ok(command)
26 }
```

check goes back with its exit status and its output, with the error stream merged in, so that compiler messages reach the model (line 14).

4.3 The sandbox on Linux

A *namespace* gives a group of processes its own copy of one kernel resource, such as the mounted file systems, the process IDs or the network interfaces [3]. Mount namespaces date from Linux 2.4.19 in 2002. User namespaces, completed in Linux 3.8 in 2013, let an unprivileged user create the other kinds [2, 6]. Bubblewrap, the sandbox tool used by Flatpak, builds views like Definition 4.2 from them [10].

We start each check through three standard programs, each with one job (Listing 2). GNU `timeout` sends `SIGTERM` after WallSeconds and `SIGKILL` KillAfter later. `prlimit` applies the per-process limits of Table 2. `bwrap` creates new user, mount, PID, network, IPC, UTS and cgroup namespaces and mounts the view. It runs as process 1 of the new PID namespace, so when it exits, the kernel ends every process that the check started. A new session keeps the check from typing

Table 3: Escape attempts and observed results. Rows marked T are unit tests, and `/home/user` stands for our home folder.

Check	Result
T: file name `../escaped.sh`	Refused before any write.
T: write `left.txt` in `/work`	Written, and absent from `harness_out` afterward.
T: overwrite the program	`Read-only file system`.
T: look for the project's `Cargo.toml` and `CARGO_MANIFEST_DIR`	Neither exists.
T: list the interfaces in `/proc/net/dev`	Only `lo`.
The hostile check of Section 1	`No such file or directory`, and `curl` exits with status 6.
`cat /home/user/.ssh/id_ed25519`	`No such file or directory`.
`curl http://1.1.1.1`	`Couldn't connect to server`.
`env`	Only `HOME`, `PATH`, `PWD` and `RUSTUP_HOME`.
`ps -e`, then `kill -9 -1`	3 processes: `bwrap`, `sh` and `ps`. The signal finds no process.
`dd` of 100 MiB into `/work`	`File size limit exceeded` at 64 MiB.
`sleep 300`	Ended by `timeout` after 120 s, with status 124.

into our terminal.

We call bubblewrap as a program, instead of creating the namespaces from Rust, because the system calls have pitfalls that bubblewrap already handles. For example, `unshare` with `CLONE_NEWPID` does not move the caller into the new PID namespace, and only its later children enter it. A sandbox that missed this would leave the check in the host's PID namespace, where `kill -9 -1` reaches every process of the user.

Some hosts do not allow unprivileged user namespaces. Ubuntu 24.04 blocks them by default through `kernel.apparmor_restrict_unprivileged_userns=1`, and the probe at the start of Procedure 1 detects this before the first request.

5 Evaluation

We measured on an ARM64 board with Linux 5.15.148, Ubuntu 22.04 and bubblewrap 0.6.1. The model, `Qwen3.6-35B-A3B`, ran on another machine behind a chat API. Each experiment is a program in our repository, and each run is saved with its commit and kernel version [7].

Escape attempts. We first wrote 12 escapes by hand, 5 of them as unit tests (Table 3). None escaped, and the 14 unit tests cover 93% of the lines and all of the sandbox. A hand-written list, however, reaches each target through one path with one program. We therefore generated variations. Each attempt reads a secret file, creates a file, or connects to a listener on the host. A file is named in 4 ways: by its plain path, through `/proc/1/root`, through `/proc/self/root`, or by climbing out of `/work` with `...` With 6 programs that read, 6 that write, 2 folders, and 4 clients with 2 host names, this gives 104 attempts.

Table 4: Two checks that stay within every per-process limit, run in the sandbox alone and in the same sandbox inside a cgroup of 64 tasks and 128 MiB with no swap.

Check	Sandbox	Sandbox in a cgroup
Start 500 sleeping processes	500 started	59 started
Write 6 files of 60 MiB to `/work`	360 MiB held in memory	Stopped by `SIGKILL` after 144 ms

A check controls its own output and exit status, so we judged each attempt on the host. A read escaped if the secret appeared in the output, a write if the target existed afterwards, and a connection if our listener accepted one. To make sure that this judge detects real escapes, we also ran every attempt as a plain shell outside the sandbox. Outside, 56 of the 104 attempts escaped. Inside, none did.

Processes and memory. The limits of `prlimit` apply to each process separately, which leaves two gaps. A check can start many processes, each within its limits, and it can fill `/work` with many files below FileSize, all held in memory. We ran both as checks, in the sandbox and in the same sandbox inside a cgroup started by `systemd-run` (Table 4). Without the cgroup, all 500 processes started and `/work` held 360 MiB. With it, 59 processes started, and the kernel stopped the memory check with `SIGKILL`. Swap must be disabled for the group, or the kernel moves the files to swap and the write succeeds.

Cost. The empty check `true` took 3 ms without the sandbox and 18 ms with it, as the mean of 150 runs, so the sandbox costs about 15 ms. Compiling and running a 144-line Rust program took 360 ms without and 365 ms with it (20 runs). The difference is smaller here because the sandbox compiles into memory. With the plain run also in memory, at 349 ms, the sandbox again adds about 15 ms, or 4%.

Hosts. On an Ubuntu 24.04 host with the AppArmor setting above, `bwrap` failed with `loopback: Failed RTM_NEWADDR: Operation not permitted`, and the harness exited before its first request.

A real task. We asked for a Rust program that demonstrates `std::mem::replace`, `std::mem::take` and the other memory functions of the standard library. The first run failed all 5 attempts with `EOF while parsing a string`. Every reply reached the model's output limit inside the program, and each retry was as long as the one before, so the failure messages could not help. A later run passed on its first attempt. Its program compiled with a warning, and a comment called `std::mem::swap` atomic, which is false. Its check tested neither.

6 Limitations

The largest open problem is the check itself. The model writes it, so a check such as `echo ok` passes any program. We plan to replace the shell command with a typed description, from which the harness builds the command. The model would then choose what to test, and the harness would decide how the test runs.

Three gaps in confinement remain in the harness. Check output returns to the model unchanged, so a program can print instructions for the next attempt. We load no seccomp filter [5], so every system call that a user namespace allows, and every kernel bug behind one, stays reachable. The harness sets no limit on processes or on total memory. The cgroup of Table 4 closes both in our tests, but it requires systemd to delegate the `pids` and `memory` controllers to the user. Finally, we measured on one board with one model.

7 Related work

ReAct interleaves reasoning with actions, Toolformer teaches a model when to call an API, and CodeAct and SWE-agent let a model run code and tests [8, 11–13]. These systems study what a model can accomplish with tools. Our harness is such a loop reduced to fixed steps, so that every effect of a reply on the host can be listed and confined. Srinivasan describes runtime patterns for production agents, among them a shared state machine and a supervisor with a gate [9]. Our harness combines both, with one difference: the model writes the check that the gate runs. MetaAgent builds multi-agent systems as finite state machines whose states the system can revisit [14]. Our 5 states are fixed and written by hand, which keeps the transitions few enough to check by reading.

8 Conclusion

We built a coding harness small enough to follow every path from a model's reply to the host. Its agent loop is a fixed state machine, so the model controls only the content of each step. That content reaches the host as a file with a plain name and as a check in a throwaway namespace sandbox, at about 15 ms per check. In our tests, no generated escape left a trace on the host, and a cgroup bounded the processes and memory that the sandbox alone did not. The next step is to take the check itself away from the model.

References

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.

[2] Michael Kerrisk. Namespaces in operation, part 1: namespaces overview. LWN.net, https://lwn.net/Articles/531114/, 2013.

[3] *namespaces(7)*. Linux man-pages project, 2026. URL https://man7.org/linux/man-pages/man7/namespaces.7.html.

[4] *openat2(2)*. Linux man-pages project, 2026. URL https://man7.org/linux/man-pages/man2/openat2.2.html.

[5] *seccomp(2)*. Linux man-pages project, 2026. URL https://man7.org/linux/man-pages/man2/seccomp.2.html.

[6] *user_namespaces(7)*. Linux man-pages project, 2026. URL https://man7.org/linux/man-pages/man7/user_namespaces.7.html.

[7] Arpan Pathak. anti_ai_slop: Skills against AI slop, and a harness for model-written code. https://github.com/arpanpathak/anti_ai_slop, 2026.

[8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[9] Vasundra Srinivasan. A methodology for selecting and composing runtime architecture patterns for production LLM agents, 2026. URL https://arxiv.org/abs/2605.20173.

[10] The Bubblewrap Authors. bubblewrap: Low-level unprivileged sandboxing tool used by Flatpak and similar projects. https://github.com/containers/bubblewrap, 2026.

[11] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yaoyu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning*, 2024.

[12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[14] Yaolun Zhang, Xiaogeng Liu, and Chaowei Xiao. MetaAgent: Automatically constructing multi-agent systems based on finite state machines. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 75667–75694, 2025.